

Änderungsprotokoll — Adressverwaltung v2.1.0

Datum: September 2026

Grundlage: Clean-Code-Analyse (14 Befunde, vgl. `CleanCode_Analyse.md`)

Bearbeitung: Vollständige Umsetzung aller 14 Befunde in Backend und Frontend

Übersicht

#	Datei	Änderung	Priorität
B-01	Program.cs	Seed-Logik in DbSeeder.cs extrahiert	 Hoch
B-02	AuthController.cs , SettingsController.cs	DTOs in Dtos/ -Ordner verschoben	 Hoch
B-03	AdressenController.cs	Notification-Logik in INotificationService extrahiert	 Mittel
B-04	EmailService.cs	int.Parse → int.TryParse (kein Absturz bei ungültigem Wert)	 Hoch
B-05	EmailService.cs , appsettings.json	Schlüsselname SenderEmail → FromAddress vereinheitlicht	 Hoch
B-06	AuthController.cs	.ToLower() → ToUpperInvariant() / OrdinalIgnoreCase	 Mittel
B-07	Models/Adresse.cs	[Required] / [MaxLength] DataAnnotations hinzugefügt	 Mittel
B-08	AdressenController.cs , CitiesController.cs	Generische Basisklasse ODataCrudController<T> eingeführt	 Mittel
F-01	lib/api.ts	normalizeAuditFields -Hilfsfunktion extrahiert (DRY)	 Mittel
F-02	components/AdresseForm.tsx	Alle Formularfelder auf Controlled Inputs umgestellt	 Mittel
F-03		confirm() / alert() → eigene	 Hoch

#	Datei	Änderung	Priorität
	app/page.tsx , neues components/ConfirmDialog.tsx	ConfirmDialog -Komponente	
F-04	neues types/next-auth.d.ts	NextAuth Session-Augmentation via Module Augmentation	● Niedrig
F-05	components/AdresseForm.tsx	Tippfehler suchelaeuft → sucheLaeuft korrigiert	● Niedrig
F-06	app/page.tsx	ladeDaten mit useCallback stabilisiert	● Niedrig

Backend — Detailbeschreibung

B-01 — DbSeeder.cs (SRP)

Neu: backend/AdressverwaltungApi/DbSeeder.cs

Die Seed-Logik (Anlegen des Admin-Benutzers) wurde aus Program.cs in eine eigene statische Klasse DbSeeder extrahiert. Zugangsdaten werden jetzt aus appsettings.Development.json gelesen statt als Magic Strings hart codiert.

```
// Program.cs: Aufruf vereinfacht auf eine Zeile
using var scope = app.Services.CreateScope();
DbSeeder.Seed(scope.ServiceProvider.GetRequiredService<AdresseDbContext>(),
               scope.ServiceProvider.GetRequiredService<IPasswordHasher<User>>(),
               app.Configuration);
```

B-02 — DTOs in eigenen Dateien (SRP)

Neu:

- backend/AdressverwaltungApi/Dtos/AuthDtos.cs — RegisterRequest , LoginRequest
- backend/AdressverwaltungApi/Dtos/SettingsDtos.cs — SettingsUpdateRequest

DTOs wurden aus den Controller-Dateien entfernt und in einen eigenen Dtos/ -Ordner verschoben.

B-03 — INotificationService (SRP)

Neu:

- backend/AdressverwaltungApi/Services/INotificationService.cs
- backend/AdressverwaltungApi/Services/EmailNotificationService.cs

Die Notification-Logik wurde aus `AdressenController` in einen eigenen Service extrahiert. Der Controller injiziert nun `INotificationService` statt direkt `IEmailService`.

```
// AdressenController.cs (vorher)
private async Task SendNewAdresseNotificationAsync(Adresse adresse) { ... }

// AdressenController.cs (nachher)
try { await _notificationService.NotifyNewAdresseAsync(adresse); }
catch (Exception ex) { _logger.LogWarning(...); }
```

B-04 — int.TryParse statt int.Parse

Geändert: backend/AdressverwaltungApi/Services/EmailService.cs

```
// Vorher (Absturz bei ungültigem Wert):
var smtpPort = int.Parse(_config["SmtpSettings:Port"] ?? "587");

// Nachher (sicher mit Fallback):
if (!int.TryParse(_config["SmtpSettings:Port"], out var smtpPort))
    smtpPort = 587;
```

B-05 — Schlüsselname FromAddress vereinheitlicht

Geändert: EmailService.cs, appsettings.json, docker-compose.yml

```
// Vorher (inkonsistent):
var senderEmail = _config["SmtpSettings:SenderEmail"] ?? smtpUser;

// Nachher (einheitlich):
var senderEmail = _config["SmtpSettings:FromAddress"] ?? smtpUser;
```

B-06 — Culture-sicherer String-Vergleich

Geändert: backend/AdressverwaltungApi/Controllers/AuthController.cs

```
// Vorher (Locale-abhängig):
u.Email.ToLower() == req.Email.ToLower()

// Nachher (culture-unabhängig, in LINQ/EF Core):
u.Email.ToUpperInvariant() == req.Email.ToUpperInvariant()
```

B-07 — DataAnnotations auf Adresse.cs

Geändert: backend/AdressverwaltungApi/Models/Adresse.cs

Alle Properties mit `[Required]` und `[MaxLength]` -Attributen versehen (vorher nur in `OnModelCreating` definiert).

B-08 — Generische Basisklasse ODataCrudController

Neu: backend/AdressverwaltungApi/Controllers/ODataCrudController.cs

```
public abstract class ODataCrudController<TEntity> : ODataController
    where TEntity : class
{
    protected readonly AdresseDbContext _context;
    protected abstract DbSet<TEntity> Entities { get; }
    // GET, POST, PATCH, DELETE gemeinsam implementiert
}

// Abgeleitete Controller:
public class AdressenController : ODataCrudController<Adresse> { ... }
public class CitiesController : ODataCrudController<City> { ... }
```

Frontend — Detailbeschreibung

F-01 — normalizeAuditFields (DRY)

Geändert: frontend/lib/api.ts

Hilfsfunktion `normalizeAuditFields` extrahiert, die die identische Audit-Feld-Normalisierung aus `normalizeAdresse` und `normalizeCity` zusammenfasst.

F-02 — Controlled Inputs in AdresseForm (UI-Konsistenz)

Geändert: frontend/components/AdresseForm.tsx

Alle Formularfelder wurden von `defaultValue` (Uncontrolled) auf `value` + `onChange` + `useState` (Controlled) umgestellt.

F-03 — ConfirmDialog statt confirm() / alert()

Neu: frontend/components/ConfirmDialog.tsx

Geändert: frontend/app/page.tsx

```
// Vorher:
if (!confirm('Adresse wirklich löschen?')) return;

// Nachher:
const [zuLoeschendId, setZuLoeschendId] = useState<number | null>(null);
// <ConfirmDialog /> rendert bei zuLoeschendId !== null
```

F-04 — NextAuth Session-Augmentation

Neu: frontend/types/next-auth.d.ts

```
declare module 'next-auth' {
  interface Session {
    user: { id: string; name?: string | null; email?: string | null; };
  }
}
```

F-05 — Tippfehler sucheLaeuft

Geändert: frontend/components/AdresseForm.tsx

```
// Vorher: const [suchelaeuft, setSuchelaeuft] = useState(false);
// Nachher:
const [sucheLaeuft, setSucheLaeuft] = useState(false);
```

F-06 — useCallback für ladeDaten

Geändert: frontend/app/page.tsx

```
// Vorher (neue Referenz bei jedem Render):
async function ladeDaten() { ... }

// Nachher (stabile Referenz):
const ladeDaten = useCallback(async () => { ... }, []);
```

Neue Dateien (Zusammenfassung)

backend/AdressverwaltungApi/	
DbSeeder.cs	← B-01 Seed-Logik
Dtos/	
AuthDtos.cs	← B-02 DTOs
SettingsDtos.cs	← B-02 DTOs
Services/	
INotificationService.cs	← B-03 Interface
EmailNotificationService.cs	← B-03 Implementierung
frontend/	
components/	
ConfirmDialog.tsx	← F-03 Dialog-Komponente
types/	
next-auth.d.ts	← F-04 Session-Augmentation

Docker: Backend mit neuem Code starten

Da Backend-Quellcode geändert wurde, muss Docker das Image neu bauen:

```
cd /home/ubuntu/adressverwaltung

# Variante 1: Rebuild und Start (Standard)
docker compose up --build

# Variante 2: Falls Datenbankvolume inkompatibel (Migrationen geändert)
docker compose down -v && docker compose up --build
```

Der `--build`-Flag ist zwingend, damit Docker die C#-Änderungen kompiliert. Ohne `--build` wird das gecachte alte Image verwendet.

Tutorial-Dokumentation

Das Tutorial wurde parallel zu den Code-Änderungen auf Version 2.0 aktualisiert:

- **Datei:** `Tutorial_WebApp_v2.0.pdf` (98 Seiten, ~16'500 Wörter)
- **Neu:** Kap 4.6–4.10 (Clean Code Backend-Änderungen), Kap 5.2–5.6 (Clean Code Frontend), Kap 14 (Clean Code & Refactoring — neues Abschlusskapitel)
- **Aktualisiert:** Cover, Inhaltsverzeichnis, Kap 9.1 (Culture-sicherer Vergleich), Kap 10.1 (INotificationService, int.TryParse)